

# Vehicle SmartPick — Databricks Onboarding Guide

<b>Repository:</b> htzd-vehicle-smartpick-1

<b>Audience:</b> New developers (e.g. datalab / labs-v24)

<b>Last updated:</b> June 2026

## 1. What This Repo Does

Vehicle SmartPick is an ML pipeline that trains and serves a <b>Top-K SIPP (car class) recommendation model</b> for Hertz online booking. It re-ranks vehicles on the search results page — it does not change inventory or pricing.

Phase | Where | What happens

**\*\*Training\*\*** (weekly) | Databricks Job | Read Amplitude events → build labeled dataset → train XGBoost → register in MLflow → update serving endpoint

**\*\*Inference\*\*** (real-time) | Databricks Model Serving | BFF calls HTTP endpoint → model returns scores → BFF sorts vehicles

## 2. Databricks Is More Than a Data Warehouse

Databricks is a <b>platform</b> with several layers SmartPick uses together:

Layer | SmartPick use

**\*\*Data / lakehouse\*\*** | Delta tables in Unity Catalog (Amplitude events)

**\*\*Compute (Spark)\*\*** | Build training choice set from billions of rows

**\*\*Jobs\*\*** | Weekly retrain (`htz-vehicle-smartpick-v24-model-train`)

**\*\*MLflow\*\*** | Experiment tracking + model registry in Unity Catalog

**\*\*Model Serving\*\*** | Real-time inference API for the BFF

<b>Mental model:</b> Data warehouse + compute + ML lifecycle + real-time serving on one platform.

## 3. Terminology Cheat Sheet

Term | Plain English

**\*\*Workspace\*\*** | Databricks environment (datalab, dev1, uat1, prd1)

**\*\*Unity Catalog (UC)\*\*** | Governance: catalogs → schemas → tables / volumes / models

**\*\*Delta table\*\*** | Columnar storage (Parquet + transactions)

**\*\*Catalog / schema\*\*** | e.g. `datalabs.lab\_shop\_and\_book`

**\*\*Job\*\*** | Scheduled or on-demand workflow

**\*\*Cluster\*\*** | Compute running Spark / Python

**\*\*MLflow\*\*** | Tracks experiments; stores model versions

**\*\*Model Serving endpoint\*\*** | Hosted API running `predict()`

**\*\*Bundle\*\*** | IaC via `databricks.yml` + `resources/\*.yaml`

**\*\*Volume\*\*** | UC-managed file storage (MLflow artifacts)

## 4. Environments & Workspace Mapping

| UI workspace name               | Bundle target(s)                | Purpose              |
|---------------------------------|---------------------------------|----------------------|
| <b>**datalab-dataplatform**</b> | <code>`labs`, `labs-v24`</code> | Sandbox – start here |
| <b>**dev-dataplatform**</b>     | <code>`dev1`, `dev1-v24`</code> | Dev integration      |
| <b>**hdp-uat1**</b>             | <code>`uat1`, `uat1-v24`</code> | Pre-prod             |
| <b>**prod-dataplatform**</b>    | <code>`prd1`, `prd1-v24`</code> | Production           |

<b>V2.4 (current path):</b> use \*-v24 targets. Model name: htz-vehicle-smartpick-v24.

### ***labs-v24 key config***

| Setting               | Value                                                                           |
|-----------------------|---------------------------------------------------------------------------------|
| Workspace             | <code>`https://hertz-datalab-dataplatform.cloud.databricks.com/`</code>         |
| Bundle code path      | <code>`~/Workspace/Users/&lt;you&gt;/.bundle/vehicle-smartpick/labs-v24`</code> |
| Events table          | <code>`datalabs.lab_shop_and_book.amplitude_from_s3_events_raw`</code>          |
| Training output table | <code>`datalabs.lab_shop_and_book.vehicle_smartpick_v24_inference`</code>       |
| Model / endpoint      | <code>`htz-vehicle-smartpick-v24`</code>                                        |
| min_event_date (labs) | <code>`2026-05-08`</code> (shorter window for faster runs)                      |

## 5. Local Development vs Databricks

### ***Works locally***

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
brew install libomp # macOS – required for XGBoost

cd src && python -m pytest ../tests/ -v
make format && make lint
```

### ***Requires Databricks***

- Full training pipeline (``src/sipp_recommendation.py``)
- Bundle deploy / run
- Model registration and endpoint updates

## 6. CLI Setup & Deploy

```
# Install CLI (Homebrew – requires tap)
brew tap databricks/tap && brew trust databricks/tap && brew install databricks

# Or manual install to ~/.local/bin (see setup-cli release)

# Authenticate
databricks auth login --host https://hertz-datalab-dataplatform.cloud.databricks.com
```

```
# Validate & deploy
cd htzd-vehicle-smartpick-1
databricks bundle validate --target labs-v24
databricks bundle deploy --target labs-v24
```

## ***Volume already exists error***

Deploy uploads code to **<b>your user path</b>**, but UC volumes are **<b>shared</b>**. If you see:

Volume 'datalabs.lab\_shop\_and\_book.htz-vehicle-smartpick-v24' already exists

Bind the existing volume, then redeploy:

```
databricks bundle deployment bind htz-vehicle-smartpick-volume \
  datalabs.lab_shop_and_book.htz-vehicle-smartpick-v24 \
  --target labs-v24
```

```
databricks bundle deploy --target labs-v24
```

Resource | Personal to you?

Bundle code upload (`root\_path`) | Yes

UC volume, model, endpoint, job | No – shared in datalabs

## **7. Where Training Data Lives (UI Walkthrough)**

**<b>Workspace:</b>** datalab-dataplatform → **<b>Catalog Explorer</b>**

```

datalabs
  ■■■ lab_shop_and_book
    ■■■ amplitude_from_s3_events_raw      ← RAW input (V2.4)
    ■■■ vehicle_smartpick_v24_inference   ← Job output (labeled rows)
    ■■■ htz-vehicle-smartpick-v24         ← Registered model + volume

```

### ***Raw events table***

- **\*\*What:\*\*** Amplitude clickstream from S3 (~30M rows/day)
- **\*\*Preview:\*\*** `event\_type`, `event\_time`, `session\_id`, `event\_properties` (JSON)
- **\*\*Labs window:\*\*** from `2026-05-08` only

### ***Intermediate training table***

Each row = one impression in a session: vehicle shown, position, clicked?, booked?

The job joins three event types via session\_id:

1. **<b>Impressions</b>** — Item List Returned
2. **<b>Clicks</b>** — Added to Cart (Vehicles)
3. **<b>Bookings</b>** — reservation API success

## **8. Running the Training Job**

**<b>Workflows → Jobs → htz-vehicle-smartpick-v24-model-train → Run now</b>**

Or CLI:

```
databricks bundle run htz-vehicle-smartpick-v24-model-train --target labs-v24
```

## ***Cluster***

- Runtime: `16.4.x-cpu-ml`
- Node: `r5d.2xlarge`, single-node (0 workers)

## ***Pipeline stages (~30–60 min in labs)***

1. **Startup** — Parse args, Spark session
2. **Build choice set** (~10–15 min) — Spark SQL → writes `vehicle_smartpick_v24_inference`
3. **Train/test split** — Last 3 days holdout; 300K / 100K session samples
4. **Feature engineering** — Trip duration, price, SIPP features, encodings
5. **Train XGBoost** — Click model + IPW bias correction
6. **Evaluate** — Hit@K, RPD, entropy → MLflow
7. **Register model** — SmartPickV24Model → UC registry, promote Champion
8. **Update endpoint** — `src/loader/endpoint/serving.py`

## ***Log lines to watch***

- `Starting V24 training`
- `Building V2.4 choice set`
- `Train: ... rows`
- `Registering model and updating serving endpoint`
- `V24 training pipeline complete`

## ***After success — where to look***

UI location | What

**MLflow** → Experiments | Metrics, artifacts

**Catalog** → Models → `htz-vehicle-smartpick-v24` | New version, Champion alias

**Serving** → `htz-vehicle-smartpick-v24` | Endpoint Ready, model version

**Catalog** → `vehicle_smartpick_v24_inference` | Labeled training rows

# **9. Endpoint Creation & BFF Consumer**

## ***Code that creates/updates the endpoint***

Call chain:

```
Job → src/sipp_recommendation.py → load() in src/loader/__init__.py  
    → register model → promote Champion  
    → update_serving_endpoint() in src/loader/endpoint/serving.py
```

- **First run:** `serving_endpoints.create()`
- **Retrain:** `serving_endpoints.update_config()` + AI Gateway re-assert

- **Endpoint name:** `args.model\_name` → `htz-vehicle-smartpick-v24`

## ***BFF is the runtime consumer (not in this repo)***

User on hertz.com

- BFF (vehicle-classes service)
- POST .../serving-endpoints/htz-vehicle-smartpick-v24/invocations
- SmartPickV24Model.predict() in src/models/smartpick\_v24.py
- scores 0.0-1.0 per vehicle
- BFF:  $\text{recommendation\_weight} = 2000 + (\text{score} \times 1000)$
- Sort vehicles (SmartPick band: 2000-2999)

**V2.4 request (simplified):**

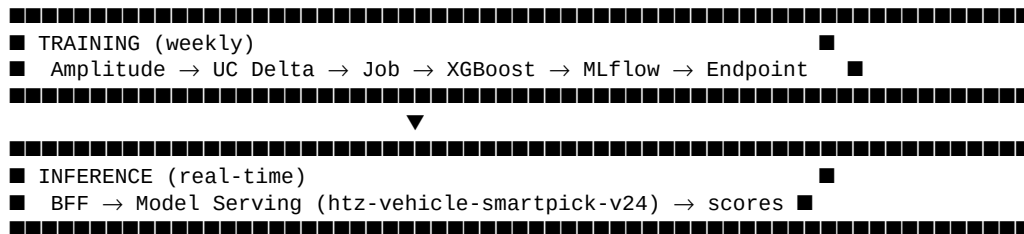
```
{
  "BRAND": "HERTZ",
  "CHKOUT_OAG": "MCOT16",
  "CHKOUT_DTTM": "2026-06-20T10:00:00",
  "CHKIN_DTTM": "2026-06-22T10:00:00",
  "TRAVL_PRPS_CD": "L",
  "VEHICLES": [
    { "sipp_code": "ICAR", "rate_type": "PAYNOW", "unit_price": 45.50 },
    { "sipp_code": "FCAR", "rate_type": "PAYNOW", "unit_price": 62.00 }
  ]
}
```

**Response:**

```
{
  "vehicles": [
    { "sipp_code": "FCAR", "score": 0.82 },
    { "sipp_code": "ICAR", "score": 0.61 }
  ]
}
```

If endpoint fails or exceeds ~200ms p99, BFF falls back to original vehicle order.

## **10. Architecture Diagram**



## **11. Project Structure**

Path | Role

`src/sipp\_recommendation.py` | Job endpoint

`src/extractor/` | Args, Spark, UC reads

`src/transformer/` | Choice set, features, train, evaluate

```
`src/loader/` | MLflow register, promote, serving update
`src/models/smartpick_v24.py` | V2.4 inference wrapper
`src/loader/endpoint/serving.py` | Endpoint create/update
`databricks.yml` | Bundle targets & env vars
`resources/model_train_v24.job.yml` | V2.4 job definition
```

## 12. Quick Checklist for New Developers

- [ ] Databricks UI login (datalab-dataplatform)
- [ ] CLI installed + `databricks auth login`
- [ ] `bundle validate --target labs-v24` passes
- [ ] Bind UC volume if deploy fails with RESOURCE\_ALREADY\_EXISTS
- [ ] `bundle deploy --target labs-v24` succeeds
- [ ] Job visible in Workflows
- [ ] (Optional) Run job — allow ~1 hour
- [ ] Verify model version + Serving endpoint in UI

## 13. What Kind of ML Training Is This?

Yes — this is **real supervised ML**, not just SQL analytics.

Aspect | SmartPick approach  
Problem | Ranking / recommendation (learned via click prediction)  
Algorithm | **XGBoost** (gradient boosted decision trees)  
Task | Binary classification: P(user clicks this vehicle)  
Label | `was\_clicked` (0 or 1)  
Features | 20 numeric/categorical features (see `src/constants.py`)  
Serve-time output | Blended score: click probability + revenue (RPD)

**Pipeline in plain English:**

1. Build labeled dataset (Spark SQL) — "vehicle shown → clicked/booked?"
2. Feature engineering (pandas) — price context, SIPP type, trip context
3. Train XGBoost on was\_clicked with IPW position-bias correction
4. Evaluate Hit@K, RPD, Shannon entropy
5. At inference: score = blend\_alpha × P(click) + (1 - alpha) × normalized\_RPD
6. Register in MLflow and update serving endpoint

**Key files:** constants.py → training.py → evaluation.py → smartpick\_v24.py

## 14. Learning Materials — Tier 1 (Core ML)

**Goal:** Labels, train/test split, classification, XGBoost, features, pandas.

Topic | Resource

ML fundamentals | [Google ML Crash Course](https://developers.google.com/machine-learning/crash-course)  
Classification & AUC | StatQuest YouTube: ROC and AUC, Sensitivity & Specificity  
Train/test split | StatQuest: Train/Test in ML  
XGBoost intuition | StatQuest: Gradient Boost series (Parts 1-4)  
XGBoost hands-on | [XGBoost Python docs – Getting Started](https://xgboost.readthedocs.io/en/stable/python/python\_intro.html)  
Feature engineering | \*Feature Engineering for ML\* (Zheng & Casari) – O'Reilly, Ch. 1-4, 6  
pandas | [10 Minutes to pandas](https://pandas.pydata.org/docs/user\_guide/10min.html)  
scikit-learn concepts | [scikit-learn Supervised Learning guide](https://scikit-learn.org/stable/supervised\_learning.html)  
**<b>Mini-project:</b>** Kaggle Click-Through Rate dataset — train a simple tabular click predictor.

## 15. Learning Materials — Tier 2 (SmartPick-Specific ML)

**<b>Goal:</b>** Ranking, position bias, revenue blending, MLflow lifecycle.

Topic | Resource

Learning to rank | Li & Xu survey – read sections 1-3  
Hit@K / ranking metrics | Recommender Systems Handbook Ch. 9, or ranking metrics summaries  
Position bias | Joachims et al. – "Unbiased Learning-to-Rank with Biased Feedback"  
IPW (accessible) | Python Causality Handbook – Propensity Score chapter  
Multi-objective blend | SmartPick `evaluation.py` + Pareto optimization overview  
MLflow | [Databricks MLflow docs](https://docs.databricks.com/en/mlflow/index.html)  
MLflow pyfunc | [MLflow Python Function models](https://mlflow.org/docs/latest/models/#python-function-model)  
Recommender systems | Coursera Recommender Systems Specialization (Courses 1-2)

**<b>In-repo reading order:</b>**

1. src/transformer/training.py — IPW + XGBoost
2. src/transformer/evaluation.py — Hit@K, RPD, entropy
3. src/models/smartpick\_v24.py — inference scoring
4. aidlc-docs/inception/reverse-engineering/codebase-summary.md — business context

## 16. Learning Materials — Tier 3 (Data Platform)

**<b>Goal:</b>** Spark SQL, Databricks jobs, Unity Catalog, Delta Lake.

Topic | Resource

Spark SQL | [Databricks Spark SQL guide](https://docs.databricks.com/en/sql/language-manual/index.html)  
PySpark | [Databricks PySpark tutorial](https://docs.databricks.com/en/languages/python.html)  
Delta Lake | [Delta Lake intro](https://docs.delta.io/latest/delta-intro.html)

Unity Catalog | [Databricks Unity Catalog fundamentals](https://docs.databricks.com/en/data-governance/unity-catalog/index.html)  
Databricks Jobs | [What are Databricks Jobs?](https://docs.databricks.com/en/jobs/index.html)  
Model Serving | [Databricks Model Serving overview](https://docs.databricks.com/en/machine-learning/model-serving/index.html)  
Asset Bundles | [Databricks Asset Bundles tutorial](https://docs.databricks.com/en/dev-tools/bundles/index.html)  
End-to-end ML on Databricks | Databricks ML Engineer Learning Path (free)  
**<b>Hands-on:</b>** Query `amplitude_from_s3_events_raw` in datalab SQL editor (use LIMIT + export\_date filter).

## 17. Suggested 4-Week Learning Path

Week | Focus | Activities

1 | Tier 1 | ML Crash Course + StatQuest XGBoost + pandas guide  
2 | Tier 1 + repo | Read ``constants.py``, ``training.py``, ``preprocess.py``; run unit tests  
3 | Tier 2 | Position bias (skim paper) + MLflow docs + ``evaluation.py``, ``smartpick_v24.py``  
4 | Tier 3 | Databricks UC + Bundles tutorials; explore datalab tables; trace one job run

## 18. Recommended Books

Book | Tier | Chapters to start

\*Hands-On Machine Learning\* (Géron) | 1 | Ch. 1-6 (skip deep learning initially)  
\*Feature Engineering for ML\* (Zheng & Casari) | 1 | All tabular chapters  
\*Designing Machine Learning Systems\* (Huyen) | 2-3 | Ch. 2-5, 8 (pipelines, eval, deployment)  
\*Learning Spark\* (Chambers et al.) | 3 | Ch. 1-3, 9 (Spark SQL / DataFrames)

## 19. Materials → SmartPick Code Map

Material | Maps to

Google ML Crash Course | Why labels exist (``was_clicked``)  
StatQuest XGBoost | ``src/transformer/training.py``  
Feature Engineering book | ``src/transformer/preprocess.py``, ``features/``  
Position bias / IPW | ``compute_ipw_weights()`` in ``training.py``  
Recommender eval metrics | ``src/transformer/evaluation.py``  
MLflow docs | ``src/loader/modeling/``  
Databricks UC + Bundles | ``databricks.yml``, deploy flow  
Spark SQL docs | ``src/transformer/choice_set.py``  
Model Serving docs | ``src/loader/endpoint/serving.py``



\*Generated for onboarding to htzd-vehicle-smartpick-1. See README.md and Deployment.md for authoritative deploy instructions.\*